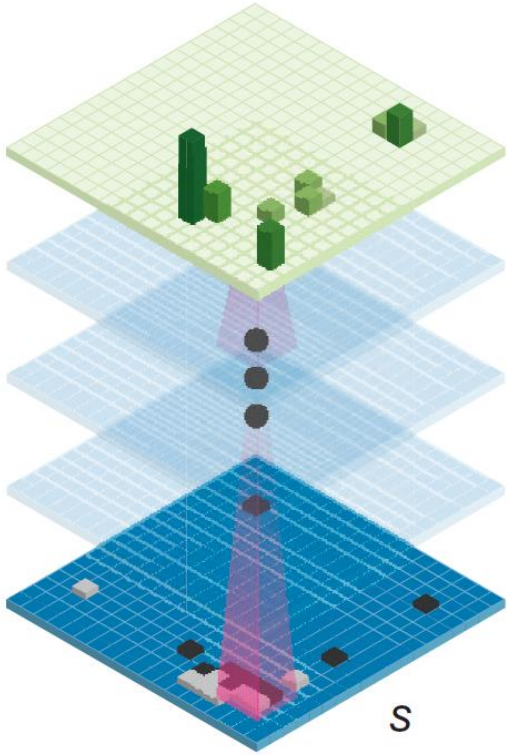


第3章 栈和队列

目录



第一节 栈的基本概念

第二节 栈的抽象数据类型描述

第三节 顺序栈与链栈

第四节 队列的基本概念

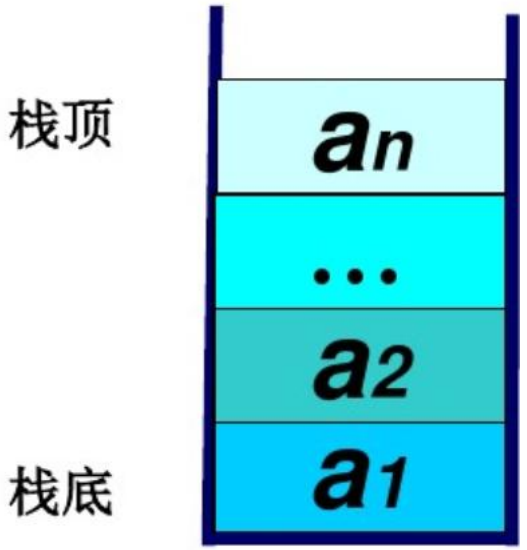
第五节 队列的抽象数据类型描述

第六节 顺序队列、链队列与优先级队列

第七节 栈和队列的比较

栈的基本概念

栈是一种特殊的线性表，其插入、删除操作只能在表的尾部进行。在栈中允许进行插入、删除操作的一端称为栈顶，另一端称为栈底。通常，栈的插入操作叫入栈，栈的删除操作叫出栈。



栈是一种特殊的线性表，其插入、删除操作只能在表的尾部进行。在栈中允许进行插入、删除操作的一端称为栈顶，另一端称为栈底。通常，栈的插入操作叫入栈，栈的删除操作叫出栈。

栈的抽象数据类型描述

```
from abc import
ABCMeta,abstractmethod,abstractproperty

class IStack(metaclass=ABCMeta):
    @abstractmethod
    def clear(self):
        '''将栈置空'''
        pass
    @abstractmethod
    def isEmpty(self):
        '''判断栈是否为空'''
        pass
    @abstractmethod
    def length(self):
        '''返回栈的数据元素个数'''
        pass
    @abstractmethod
    def peek(self):
        '''返回栈顶元素'''
        pass
    @abstractmethod
    def push(self,x):
        '''数据元素 x 入栈'''
        pass
    @abstractmethod
    def pop(self):
        '''将栈顶元素出栈并返回'''
        pass
    @abstractmethod
    def display(self):
        '''输出栈中的所有元素'''
        pass
```

栈中的数据元素和数据间的逻辑关系与线性表相同，是由n个具有相同数据类型的数据元素构成的有限序列

栈的抽象数据类型描述

栈的抽象数据Python抽象类包含了栈的主要基本操作，如果要使用这个类还需要具体的类来实现。

栈的Python抽象类的实现方法主要有以下两种：

- **基于顺序存储的实现，为顺序栈**
- **基于链式存储的实现，为链栈**

顺序栈：Python描述

```python
class SqStack(IStack):
    def __init__(self,maxSize):
        self.maxSize = maxSize # 栈的最大存储单元个数
        self.stackElem = [None] * self.maxSize # 顺序栈存储空间
        self.top = 0 # 指向栈顶元素的下一个存储单元位置

    def clear(self):
        '''将栈置空'''
        self.top = 0

    def isEmpty(self):
        '''判断栈是否为空'''
        return self.top == 0

    def length(self):
        '''返回栈的数据元素个数'''
        return self.top
```

```python
    def peek(self):
        '''返回栈顶元素'''
        if not self.isEmpty():
            return self.stackElem[self.top-1]
        else:
            return None
    def push(self,x):
        '''数据元素 x 入栈'''
        pass

    def pop(self):
        '''将栈顶元素出栈并返回'''
        pass

    def display(self):
        '''输出栈中的所有元素'''
        for i in range(self.top-1,-1,-1):
            print(self.stackElem[i],end=' ')
```

顺序栈用数组实现，因为入栈和出栈操作都是在栈顶进行，所以增加变量top来指示栈顶元素的位置，top指向栈顶元素存储位置的下一个存储单元的位置，空栈时top=0。

顺序栈的入栈操作

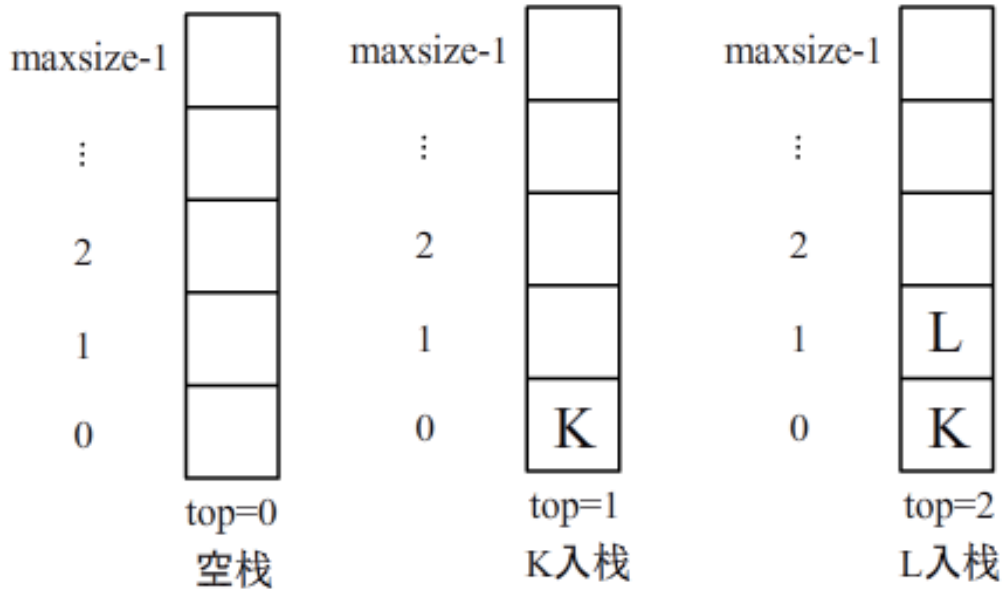
主要流程：

(1) 判断顺序栈是否为满，若满则抛出异常。

(2) 将x存入top所指的存储单元位置。

(3) top加1。

```
1  def push(self,x):
2          '''数据元素 x 入栈'''
3          if self.top == self.maxSize:
4                  raise Exception("栈已满")
5          self.stackElem[self.top] = x
6          self.top += 1
```



顺序栈的出栈操作

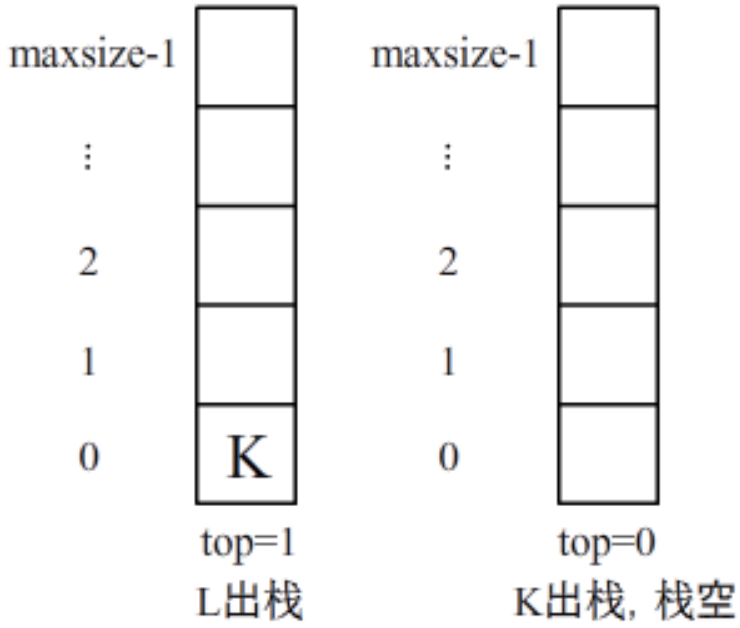
主要流程：

(1) 判断顺序栈是否为空，若空则返回null。

(2) top减1。

(3) 返回top所指的栈顶元素的值。

```
1  def pop(self):
2          '''将栈顶元素出栈并返回'''
3          if self.isEmpty():
4                  return None
5          self.top -= 1
6          return self.stackElem[self.top]
```



例题

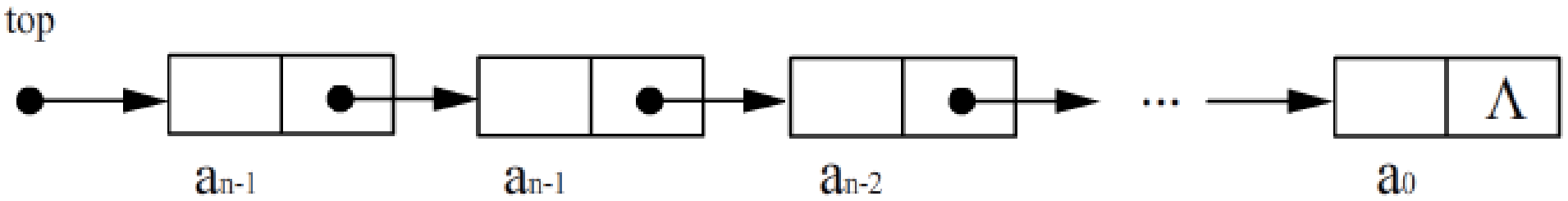
利用顺序栈实现括号匹配的语法检查。

解：
括号匹配是指程序中出现的括号，左、右括号的个数是相同的，并且需要先左后右依次出现。括号是可以嵌套的，一个右括号与其前面最近的一个左括号匹配，使用栈保存多个嵌套的左括号。

```
1  def isMatched(str):
2          s = SqStack(100)
3          for c in str:
4                  if c=='(':
5                          s.push('(')
6                  elif c==')' and not s.isEmpty():
7                          s.pop()
8                  elif c==')' and s.isEmpty():
9                          print("括号不匹配")
10                         return False
11         if s.isEmpty():
12                 print("括号匹配")
13                 return True
14         else:
15                 print("括号不匹配")
16                 return False
```

链栈

采用链式存储结构的栈称为链栈，由于入栈和出栈只能在栈顶进行，不存在在栈的任意位置进行插入和删除的操作，所以不需要设置头结点，只需要将指针top指向栈顶元素结点，每个结点的指针域指向其后继结点即可。



链栈

```
class Node(object):
    def __init__(self,data=None,next=None):
        self.data = data
        self.next = next

class LinkStack(IStack):
    def __init__(self):
        self.top = None

    def clear(self):
        '''将栈置空'''
        self.top = None

    def isEmpty(self):
        '''判断栈是否为空'''
        return self.top is None

    def length(self):
        '''返回栈的数据元素个数'''
        i = 0
        p = self.top
        while p is not None:
            p = p.next
            i += 1
        return i
```

```
    def peek(self):
        '''返回栈顶元素'''
        return self.top

    def push(self,x):
        '''数据元素 x 入栈'''
        s = Node(x,self.top)
        self.top = s

    def pop(self):
        '''将栈顶元素出栈并返回'''
        if self.isEmpty():
            return None
        p = self.top
        self.top = self.top.next
        return p.data

    def display(self):
        '''输出栈中的所有元素'''
        p = self.top
        while p is not None:
            print(p.data,end=' ')
            p = p.next
```

实现IStack抽象类的链栈类的Python语言描述

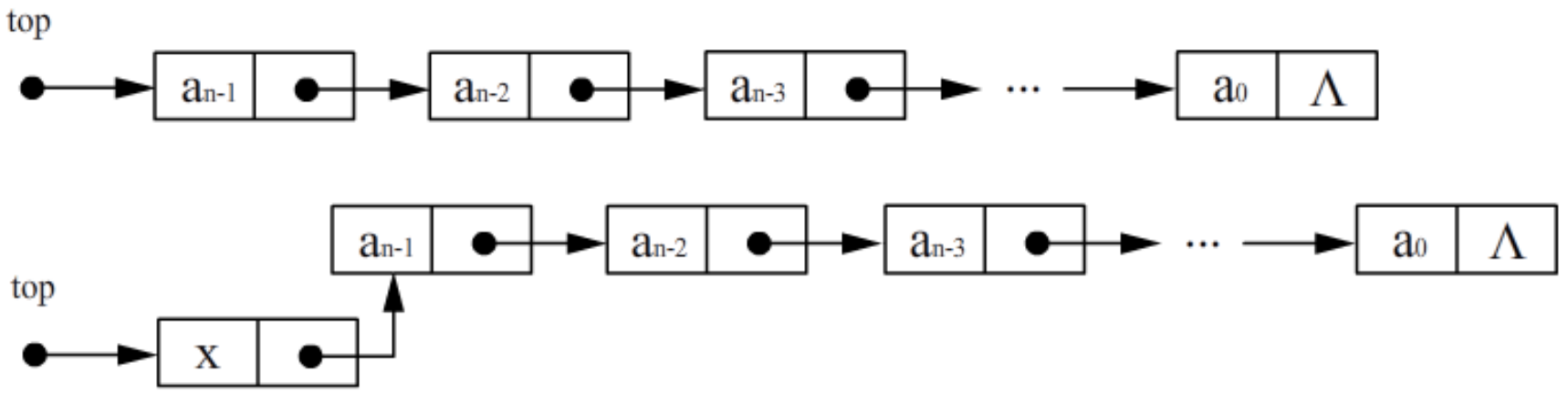
链栈的入栈操作

主要流程：

(1) 构造数据值域为x的新结点。

(2) 改变新结点和首结点的指针域，使新结点成为新的栈顶结点。

```
1  def push(self,x):
2          '''数据元素 x 入栈'''
3          s = Node(x, self.top)
4          self.top = s
```



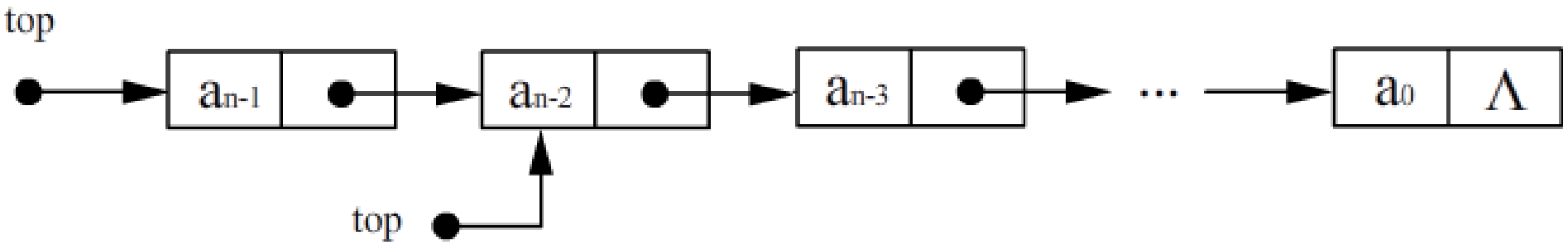
链栈的出栈操作

主要流程：

(1) 判断链栈是否为空，若空则返回null。

(2) 修改top指针域的值，返回被删结点的数据域值。

```
1  def pop(self):
2          '''将栈顶元素出栈并返回'''
3          if self.isEmpty():
4                  return None
5          p = self.top
6          self.top = self.top.next
7          return p.data
```



分析可得，使用单链表实现栈，入栈和出栈操作的实现为单链表的头插入和头删除，时间复杂度为O(1)。

例题

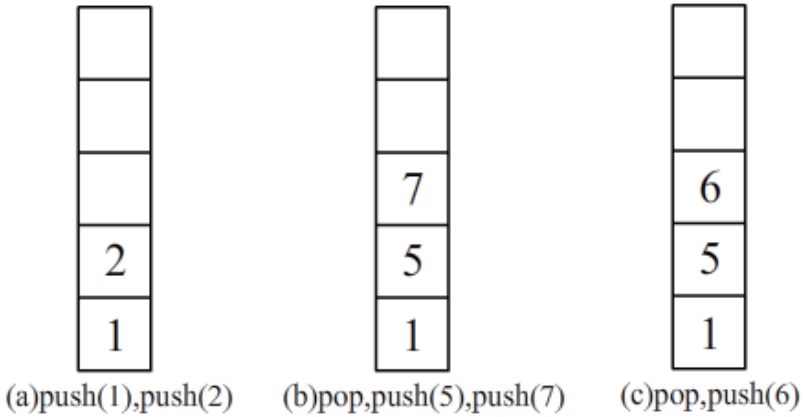
设有编号为1、2、3、4的4辆列车顺序进入一个栈式结构的车站，具体写出这4辆列车开出车站的所有可能的顺序。

解： 至少有14种。
全进之后再出的情况只有1种： 4,3,2,1。
进3个之后再出的情况有3种： 3,4,2,1； 3,2,4,1； 3,2,1,4。
进两个之后再出的情况有5种： 2,4,3,1； 2,3,4,1； 2,1, 3,4； 2,1,4,3； 2,1,3,4。
进一个之后再出的情况有5种： 1,4,3,2； 1,3,2,4； 1,3,4,2； 1, 2,3,4； 1,2,4,3。

例题

在执行操作序列push(1)、push(2)、pop、push(5)、push(7)、pop、push(6)之后栈顶元素和栈底元素分别是什么？

解：操作序列的执行过程如图所示



(a)push(1),push(2)　(b)pop,push(5),push(7)　(c)pop,push(6)

所以栈顶元素为6、栈底元素为1。

例题

编程实现汉诺塔问题的求解。假设有3个分别命名为x、y、z的塔座，在塔座x上插有n个直径和序号均为1、2、…、n的圆盘。要求将塔座x上的n个圆盘借助塔座y移动到塔座z上，仍按照相同的序号排列，并且每次只能移动一个圆盘，在任何时候都不能将一个较大的圆盘压在较小的圆盘之上。

解：分析问题可知，当n=1时只要将序号为1的圆盘从x直接移动到z即可； 当n>1时则需要将序号小于n的n-1个圆盘移动到y上，再将序号为n的圆盘移动到z上，然后将y上的n-1个圆盘移动到z上。如何将n-1个圆盘移动到z上是一个和原问题相似的问题，只是规模变小，可以用同样的方法求解。代码如右：

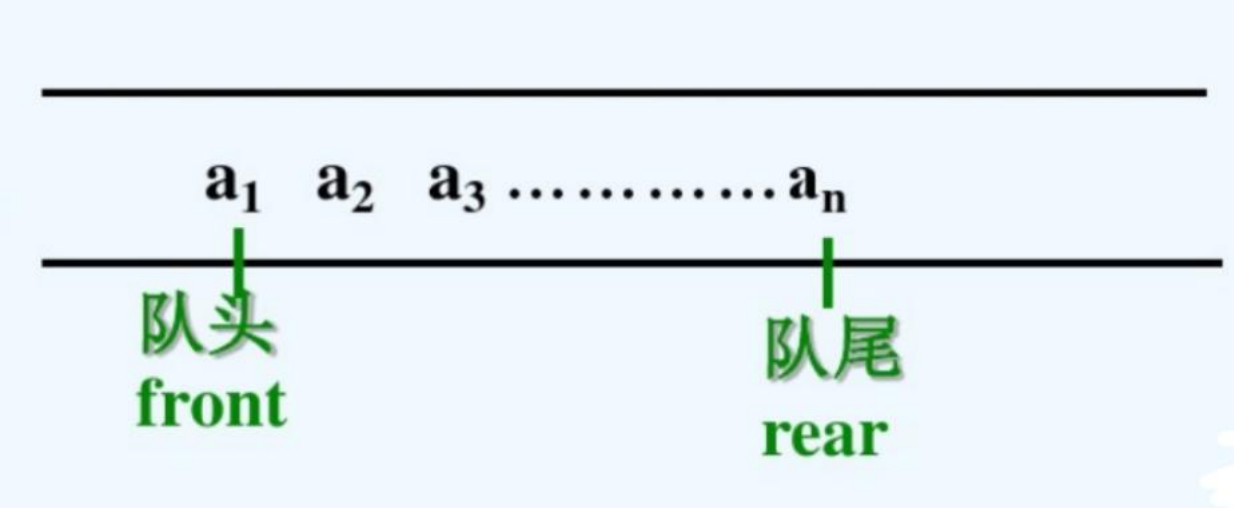
```
def move(s,t):
        print("将",s,"塔座上最顶端圆盘移动到",t,"塔座上")

def hanoi(n,x,y,z):
        if n==1:
                move(x,z)
        else:
                hanoi(n-1,x,z,y) # 将 x 上 n-1 个圆盘从 x 移动到 y
                move(x,z) # 1 个圆盘从塔座 x 移动到塔座 z
                hanoi(n-1,y,x,z) # 将 y 上 n-1 个圆盘从 y 移动到 z

hanoi(3,'x','y','z')
```

队列的基本概念

队列是一种特殊的线性表，其插入操作只能在表的尾部进行，删除操作只能在表头进行。在队列中允许进行插入操作的一端称为队尾，允许进行删除操作的另一端称为队首。通常，队列的插入操作叫入队，队列的删除操作叫出队。没有数据元素的队列称为空队列。



由于插入和删除操作分别在队尾和队首进行，最先入队的元素总是最先出队，因此队列具有先进先出的特点。

队列的抽象数据类型描述

```
from abc import ABCMeta,abstractmethod,abstractproperty

class IQueue(metaclass=ABCMeta):
        @abstractmethod
        def clear(self):
                '''将队列置空'''
                pass
        @abstractmethod
        def isEmpty(self):
                '''判断队列是否为空'''
                pass
        @abstractmethod
        def length(self):
                '''返回队列的数据元素个数'''
                pass
```

```
    @abstractmethod
    def peek(self):
            '''返回队首元素'''
            pass
    @abstractmethod
    def offer(self,x):
            '''将数据元素 x 插入到队列成为队尾元素'''
            pass
    @abstractmethod
    def poll(self):
            '''将队首元素删除并返回其值'''
            pass
    @abstractmethod
    def display(self):
            '''输出队列中的所有元素'''
            pass
```

队列中的数据元素和数据间的逻辑关系与线性表相同，是由
n个具有相同数据类型的数据元素构成的有限序列

队列的抽象数据类型描述

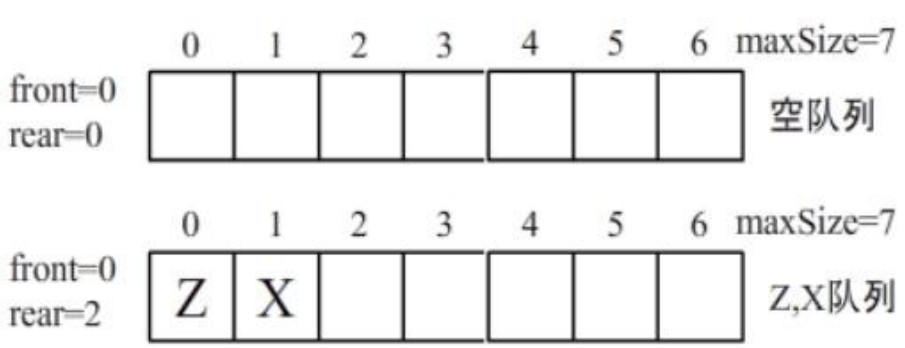
队列的抽象数据Python抽象类包含了队列的主要基本操作，如果要使用这个接口，还需要具体的类来实现。

Python抽象类的实现方法主要有以下两种：

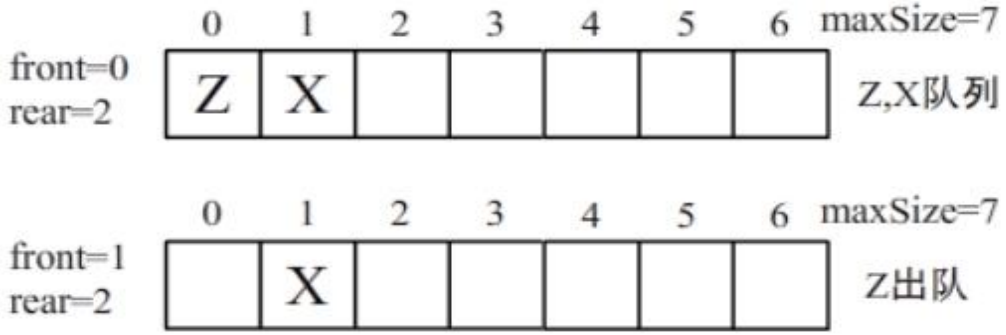
基于顺序存储的实现，为顺序队列

基于链式存储的实现，为链队列

顺序队列



入队操作



出队操作

顺序队列的存储结构与顺序栈类似，可用数组实现，因为入队和出队操作分别在队尾和队首进行，所以增加变量front来指示队首元素的位置，rear指示队尾元素的下一个存储单元的位置。

顺序队列

顺序队列类的Python语言描述如下：

```
class SqQueue(IQueue):
    def __init__(self,maxSize):
        self.maxSize = maxSize # 队列的最大存储单元个数
        self.queueElem = [None] * self.maxSize # 队列的存储空间
        self.front = 0 # 指向队首元素
        self.rear = 0 # 指向队尾元素的下一个存储单元

    def clear(self):
        '''将队列置空'''
        self.front = 0
        self.rear = 0

    def isEmpty(self):
        '''判断队列是否为空'''
        return self.rear==self.front

    def length(self):
        '''返回队列的数据元素个数'''
        return self.rear-self.front

    def peek(self):
        '''返回队首元素'''
        if self.isEmpty():
            return None
        else:
            return self.queueElem[self.front]
```

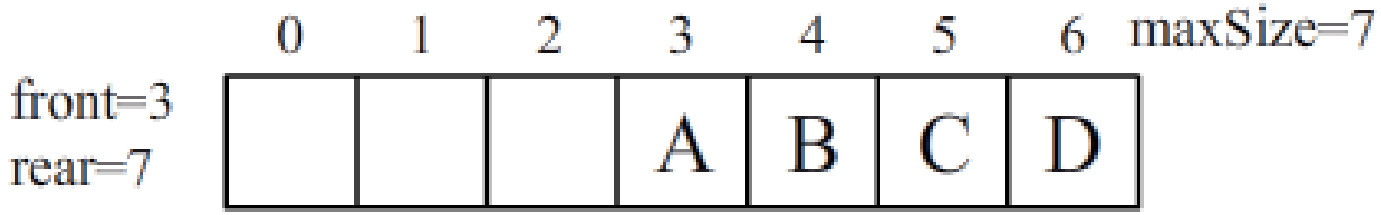
```
    def offer(self,x):
        '''将数据元素x插入到队列成为队尾元素'''
        if self.rear==self.maxSize:
            raise Exception("队列已满")
        self.queueElem[self.rear] = x
        self.rear += 1

    def poll(self):
        '''将队首元素删除并返回其值'''
        if self.isEmpty():
            return None
        p = self.queueElem[self.front]
        self.front += 1
        return p

    def display(self):
        '''输出队列中的所有元素'''
        for i in range(self.front,self.rear):
            print(self.queueElem[i],end=' ')
```

循环队列

分析发现，顺序队列的多次入队和出队操作会造成有存储空间却不能进行入队操作的“假溢出”现象。

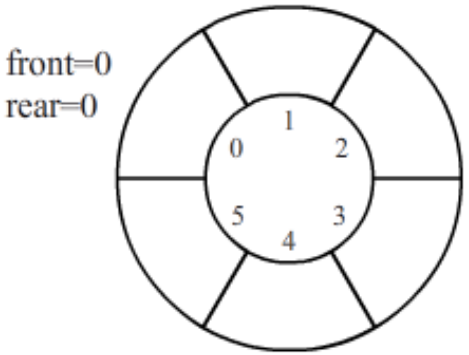


假溢出

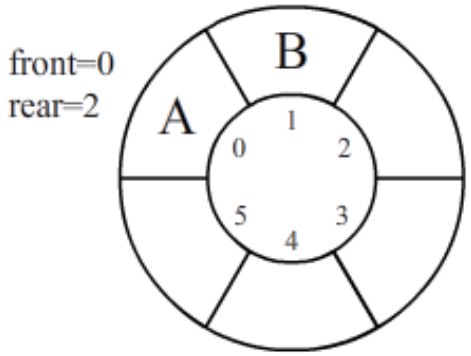
顺序队列之所以会出现“假溢出”现象是因为顺序队列的存储单元没有重复使用机制，为了解决顺序队列因数组下标越界而引起的“溢出”问题，可将顺序序列的首尾相连，形成循环顺序队列。

循环队列

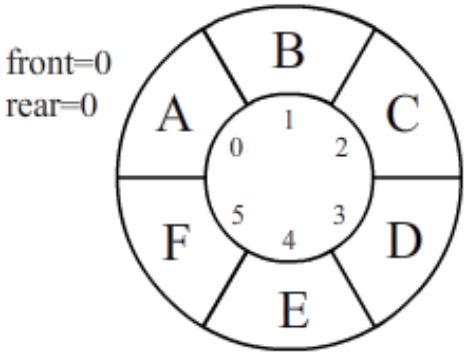
循环顺序队列进行入队和出队操作后的状态变化如图：



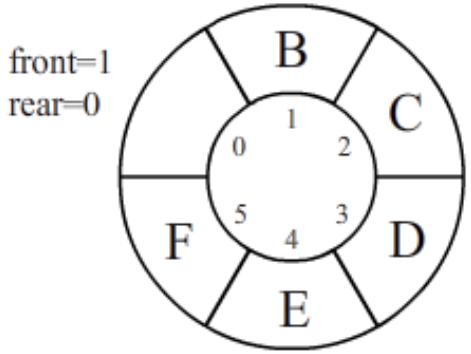
空循环队列



A,B入队



满循环队列



A出队

有新的问题产生——队空和队满的判定条件都变为front==rear，为了解决这一问题，可少利用一个存储单元，队列最多存放maxSize-1个数据元素，队空的判定条件为front==rear，队满的判定条件为front=(rear+1)%maxSize。

循环队列

```
class CircleQueue(IQueue):
        def __init__(self,maxSize):
                self.maxSize = maxSize # 队列的最大存储单元个数
                self.queueElem = [None] * self.maxSize # 队列的存储空间
                self.front = 0 # 指向队首元素
                self.rear = 0 # 指向队尾元素的下一个存储单元

        def clear(self):
                '''将队列置空'''
                self.front = 0
                self.rear = 0

        def isEmpty(self):
                '''判断队列是否为空'''
                return self.rear==self.front

        def length(self):
                '''返回队列的数据元素个数'''
                return (self.rear-self.front+self.maxSize)%self.maxSize

        def peek(self):
                '''返回队首元素'''
                if self.isEmpty():
                        return None
                else:
                        return self.queueElem[self.front]
```

循环顺序队列类和顺序队列类的Python语言描述相似，仅是指示变量front和rear的修改以及队满的判定条件发生了变化。

```
def offer(self,x):
        '''将数据元素x插入到队列成为队尾元素'''
        if (self.rear+1)%self.maxSize==self.front:
                raise Exception("队列已满")
        self.queueElem[self.rear] = x
        self.rear = (self.rear+1)%self.maxSize

def poll(self):
        '''将队首元素删除并返回其值'''
        if self.isEmpty():
                return None
        p = self.queueElem[self.front]
        self.front = (self.front+1)%self.maxSize
        return p

def display(self):
        '''输出队列中的所有元素'''
        i = self.front
        while i!=self.rear:
                print(self.queueElem[i],end=' ')
                i = (i+1)%self.maxSize
```

例题

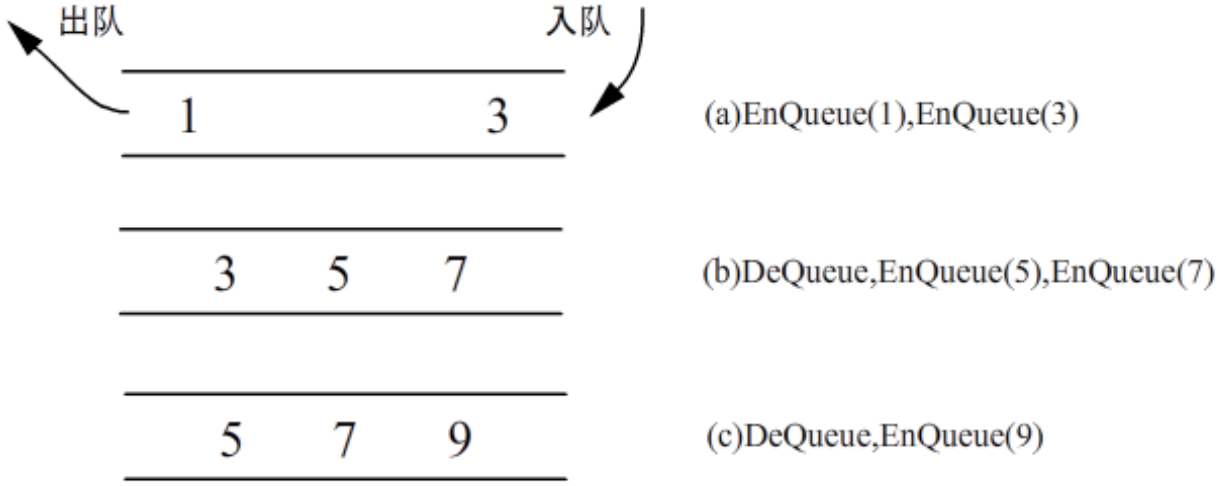
假定用于顺序存储一个队列的数组的长度为N，队首和队尾指针分别为front和rear，写出求此队列长度(即所含元素个数)的公式。

解： 当rear大于等于front时队列长度为rear-front，也可以表示为(rear-front+N)%N； 当rear小于front时队列被分成两个部分，前部分在数组尾部，其元素个数为N-1-front，后部分在数组首部，其元素个数为rear+1，两者相加为rear-front+N。综上所述，在任何情况下队列长度的计算公式都为(rear-front+N)%N。

例题

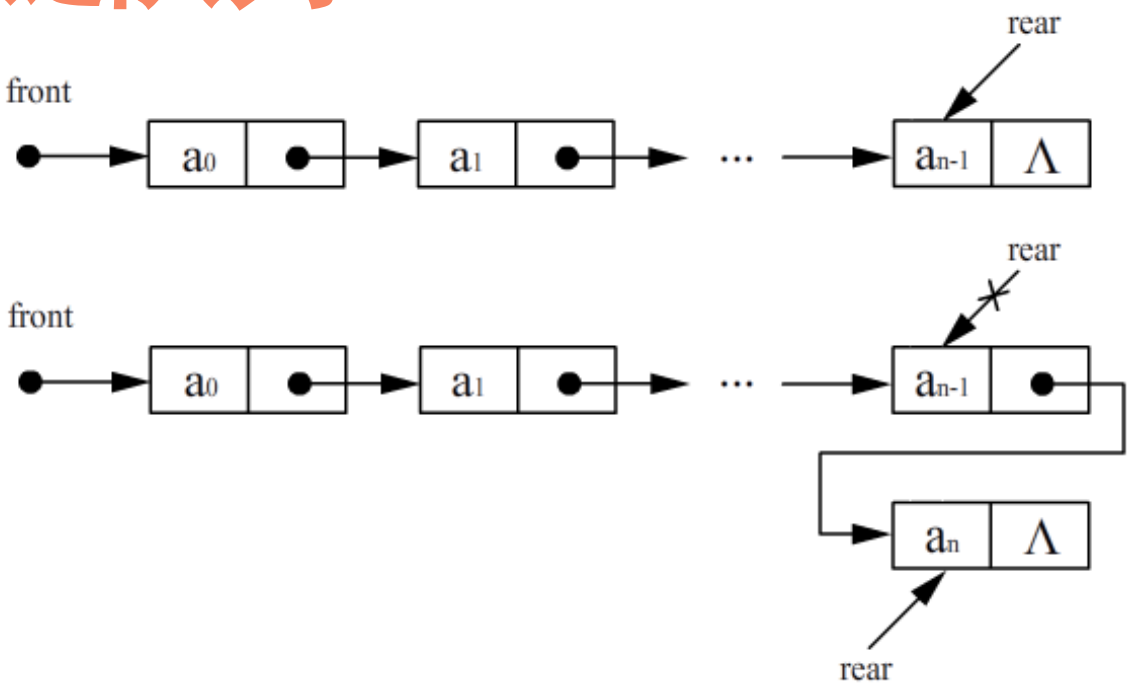
在执行操作序列EnQueue(1)、EnQueue(3)、DeQueue、EnQueue(5)、EnQueue(7)、DeQueue、EnQueue(9)之后队首元素和队尾元素分别是什么？EnQueue(k)表示整数k入队，DeQueue表示队首元素出队。

解：操作序列的执行过程如图所示

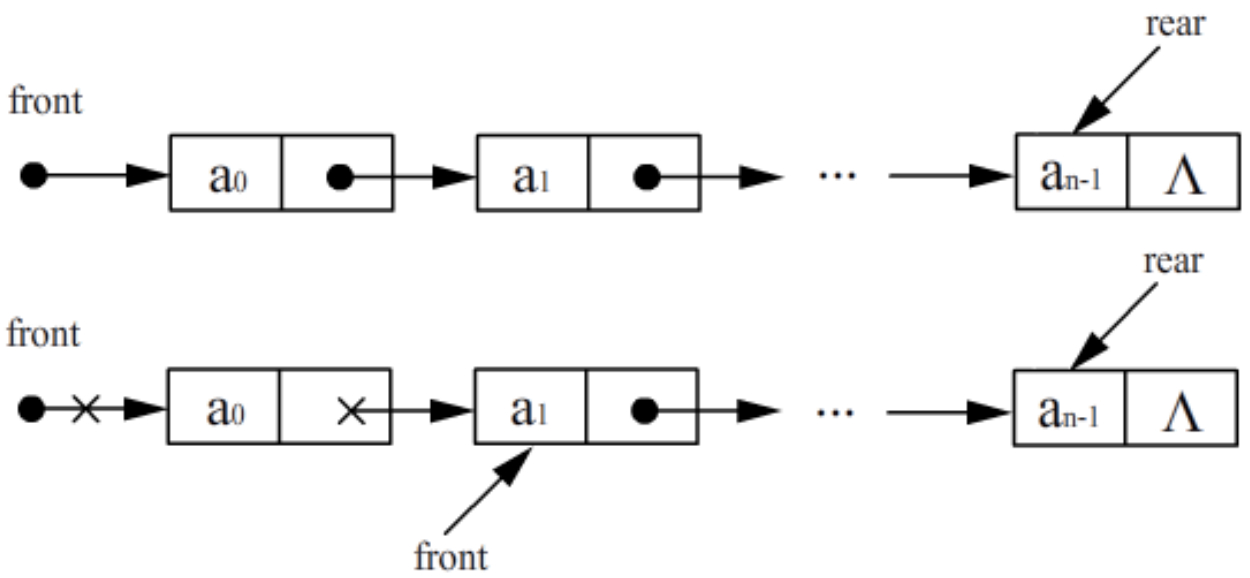


所以队首元素为5，队尾元素为9。

链队列



入队操作



出队操作

链队列用单链表实现，由于入队和出队分别在队列的队尾和队首进行，不存在在队列的任意位置进行插入和删除的情况，所以不需要设置头结点，只需要将指针front和rear分别指向队首结点和队尾结点，每个结点的指针域指向其后继结点即可。

链队列

```
class Node(object):
    def __init__(self,data=None,next=None):
        self.data = data
        self.next = next

class LinkQueue(IQueue):
    def __init__(self):
        self.front = None # 队首指针
        self.rear = None # 队尾指针

    def clear(self):
        '''将队列置空'''
        self.front = None
        self.rear = None

    def isEmpty(self):
        '''判断队列是否为空'''
        return self.front is None
```

```
def length(self):
    '''返回队列的数据元素个数'''
    p = self.front
    i = 0
    while p is not None:
        p = p.next
        i +=1
    return i

def peek(self):
    '''返回队首元素'''
    if self.isEmpty():
        return None
    else:
        return self.front.data

def offer(self,x):
    '''将数据元素 x 插入到队列成为队尾元素'''
    s = Node(x,None)
    if not self.isEmpty():
        self.rear.next = s
    else:
        self.front = s
    self.rear = s
```

```
def poll(self):
    '''将队首元素删除并返回其值'''
    if self.isEmpty():
        return None
    p = self.front
    self.front = self.front.next
    if p == self.rear: # 删除结点为队尾结点时需要修改rear
        self.rear = None
    return p.data

def display(self):
    '''输出队列中的所有元素'''
    p = self.front
    while p is not None:
        print(p.data,end=' ')
        p = p.next
```

利用Node类，链队列的Python语言描述如图。

优先级队列



有些应用中的排队等待问题仅按照“先来先服务”原则不能满足要求，还需要将任务的重要程度作为排队的依据。例如操作系统中的进程调度管理，每个进程都有一个优先级值表示进程的紧急程度，优先级高的进程先执行，同级进程按照先进先出原则排队等待，因此操作系统需要使用优先级队列来管理和调度进程。

优先级队列是在普通队列的基础之上将队列中的数据元素按照关键字的值进行有序排列。优先级队列在队首进行删除操作，但为了保证队列的优先级顺序，插入操作不一定在队尾进行，而是按照优先级插入到队列的合适位置。

和其他数据结构类似，优先级队列也可以采用顺序和链式两种存储结构。但为了快速地访问优先级高的元素以及快速地插入数据元素，通常使用链式存储结构。

优先级队列

优先级队列结点类描述

```
class PriorityNode:
    def __init__(self,data=None,priority=None,next=None):
        self.data = data # 结点的数据域
        self.priority = priority # 结点的优先级
        self.next = next
```

在此优先队级列中，数据元素的优先级别依据优先数的大小进行判定，即优先数越小优先级别越大。

优先级队列

优先级队列类的描述及实现

```
class PriorityQueue(IQueue):
        def __init__(self):
                self.front = None # 队首指针
                self.rear = None # 队尾指针

        def clear(self):
                '''将队列置空'''
                self.front = None
                self.rear = None

        def isEmpty(self):
                '''判断队列是否为空'''
                return self.front is None
```

```
def length(self):
        '''返回队列的数据元素个数'''
        p = self.front
        i = 0
        while p is not None:
                p = p.next
                i +=1
        return i

def peek(self):
        '''返回队首元素'''
        if self.isEmpty():
                return None
        else:
                return self.front.data
```

优先级队列

（续）优先级队列类的描述及实现

```
def offer(self,x,priority):
        '''将数据元素x插入到队列成为队尾元素'''
        s = PriorityNode(x,priority,None)
        if not self.isEmpty():
                p = self.front
                q = self.front
                while p is not None and p.priority<=s.priority:
                        q = p
                        p = p.next
                # 元素位置的三种情况
                if p is None: # 队尾
                        self.rear.next = s
                        self.rear = s
                elif p == self.front: # 队首
                        s.next = self.front
                        self.front = s
                else: # 队中
                        q.next = s
                        s.next = p
        else:
                self.front = self.rear = s
```

```
def poll(self):
        '''将队首元素删除并返回其值'''
        if self.isEmpty():
                return None
        p = self.front
        self.front = self.front.next
        if p == self.rear: # 删除结点为队尾结点时需要修改rear
                self.rear = None
        return p.data

def display(self):
        '''输出队列中的所有元素'''
        p = self.front
        while p is not None:
                print(p.data,end=' ')
                p = p.next
```

例题

利用优先级队列模仿操作系统的进程管理问题，要求优先级高的进程先获得CPU，优先级相同的进程先到的先获得CPU。假设操作系统中的进程由进程号和进程优先级两部分组成。

解:

```
1  pq = PriorityQueue()
2  pq.offer(1,20)
3  pq.offer(2,30)
4  pq.offer(3,10)
5  pq.offer(4,50)
6  print("进程服务的顺序为：")
7  while not pq.isEmpty():
8          print(pq.poll())
```

栈与队列的比较

相同点	（1） 均为线性结构，数据元素间具有“一对一”的逻辑关系； （2） 都有顺序存储和链式存储两种实现方式； （3） 操作受限，插入操作均在表尾进行(优先级队列除外)； （4） 插入与删除操作都具有常数时间
不同点	（1） 栈删除操作在表尾进行，具有后进先出特性； 队列的删除操作在表头进行，具有先进先出特性。 （2） 顺序栈可以实现多栈空间共享，而顺序队列则不同

小结

(1) 栈是一种特殊的线性表，它只允许在栈顶进行插入和删除操作，具有后进先出的特性，各种运算的时间复杂度为O(1)。栈采用顺序存储结构或者链式存储结构。

(3) 循环队列是将顺序队列的首尾相连，解决“假溢出”现象的发生。



(2) 队列是一种特殊的线性表，它只允许在表头进行删除操作、在表尾进行插入操作，具有先进先出的特性，各种运算的时间复杂度为O(1)。队列采用顺序存储结构或者链式存储结构。

(4) 优先级队列是在普通队列的基础之上将队列中的数据元素按照关键字的值进行有序排列。在表头进行删除操作，插入操作按照优先级插入到队列的合适位置。

谢谢观看